

Anexo_Codigo_Sucio

Título: Informe de Análisis Técnico: Identificación de Code Smells y Malas Prácticas

1. Introducción

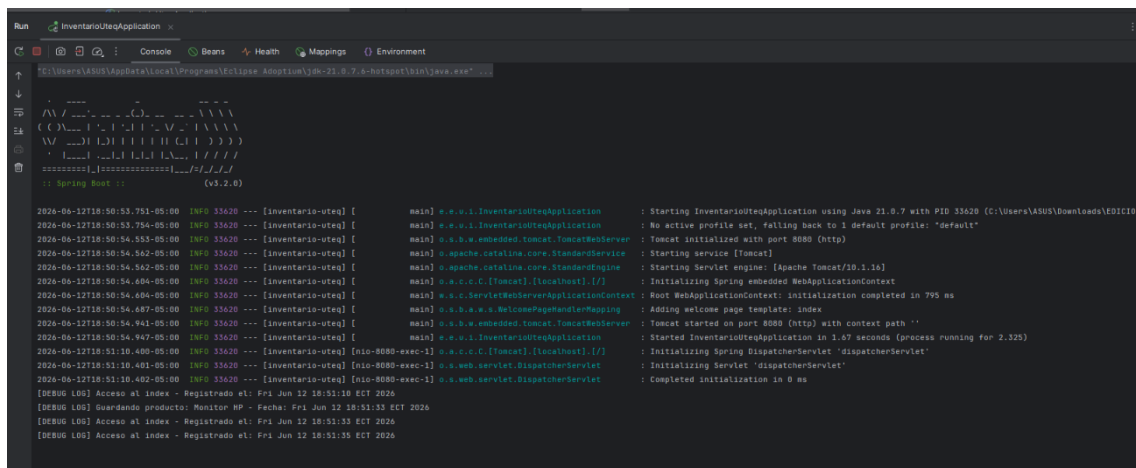
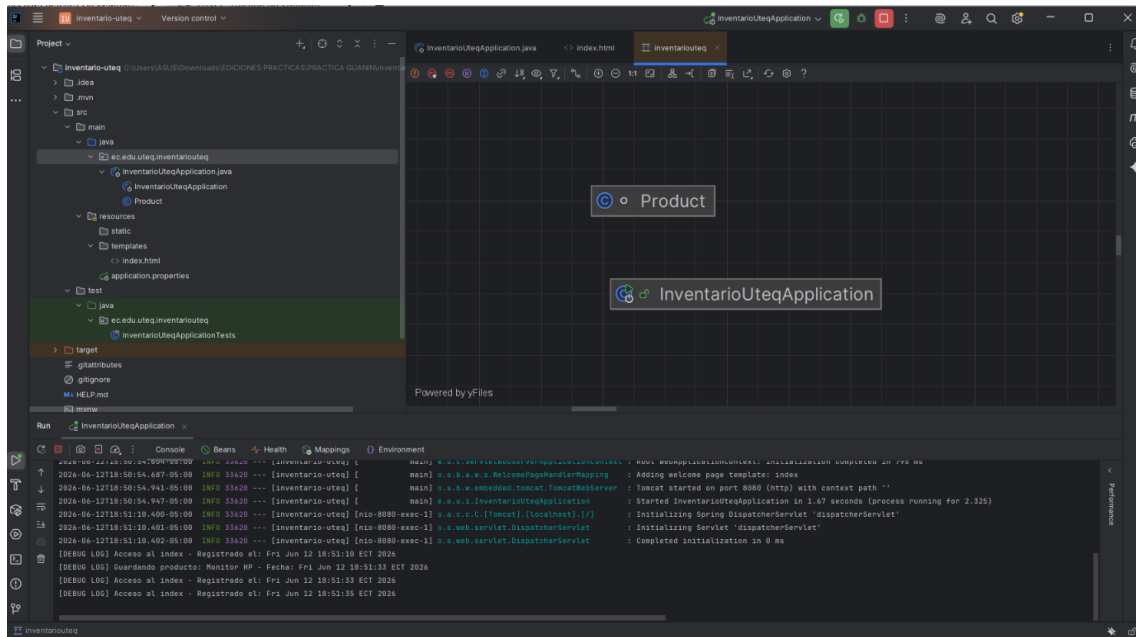
El presente documento detalla el estado inicial del "Sistema de Inventario UTEQ". En esta fase, el desarrollo se centró únicamente en la funcionalidad, descuidando la arquitectura y las buenas prácticas de programación, lo que resultó en un código difícil de mantener y escalar.

2. Identificación de Code Smells (Problemas de Código)

- **2.1. God Object (Objeto Dios):**
La clase principal `InventarioUteqApplication` asume demasiadas responsabilidades. No solo arranca la aplicación, sino que también actúa como controlador, base de datos (mediante una lista estática) y define el modelo de datos. Esto viola el Principio de Responsabilidad Única (SRP).
- **2.2. Código Duplicado (Duplicated Code):**
Se observa que las instrucciones de registro de logs (`System.out.println`) están repetidas manualmente en cada método del controlador (`index`, `guardar`, `borrar`). Esto genera redundancia y dificulta cambios globales en el sistema de mensajería.
- **2.3. Método Largo y Lógica Enredada (Long Method):**
El método `guardar` contiene lógica de negocio compleja, como la búsqueda manual de productos por ID y la actualización de campos mediante un bucle `for` y múltiples `if/else`. Esta lógica debería estar aislada en una capa de servicios.

3. Métricas del Estado "Sucio"

- **Complejidad:** Alta (Lógica concentrada).
- **Acoplamiento:** Muy Alto (El controlador depende directamente de la implementación de la lista).
- **Mantenibilidad:** Crítica (Modificar un campo del producto implica cambiar múltiples secciones de la clase principal).



Code Smell: God Object

```
@SpringBootApplication
@Controller
public class InventarioUteqApplication {
    // Aquí está el error: El controlador maneja directamente la lista
    // de datos
    private static List<Product> productos = new ArrayList<>();

    // Y también define la lógica de la aplicación
    @GetMapping("/")
    public String index(Model model) { ... }
}

// Y para colmo, la clase de datos está metida aquí mismo al final
class Product { ... }
```

Code Smell: Duplicated Code (Código Duplicado)

```
@GetMapping("/")
public String index(Model model) {
    // LOG REPETIDO
    System.out.println("[DEBUG LOG] Acceso al index - Registrado el: "
+ new java.util.Date());
```

```
@PostMapping("/guardar")
public String guardar(Product p) {
    // LOG REPETIDO
    System.out.println("[DEBUG LOG] Guardando producto: " +
p.getNombre() + " - Fecha: " + new java.util.Date());
```

```
public String borrar(@PathVariable Long id) {
    System.out.println("[DEBUG LOG] Eliminando ID: " + id + " - Fecha:
" + new java.util.Date());
```

Code Smell: Long Method (Método Largo / Lógica Enredada)

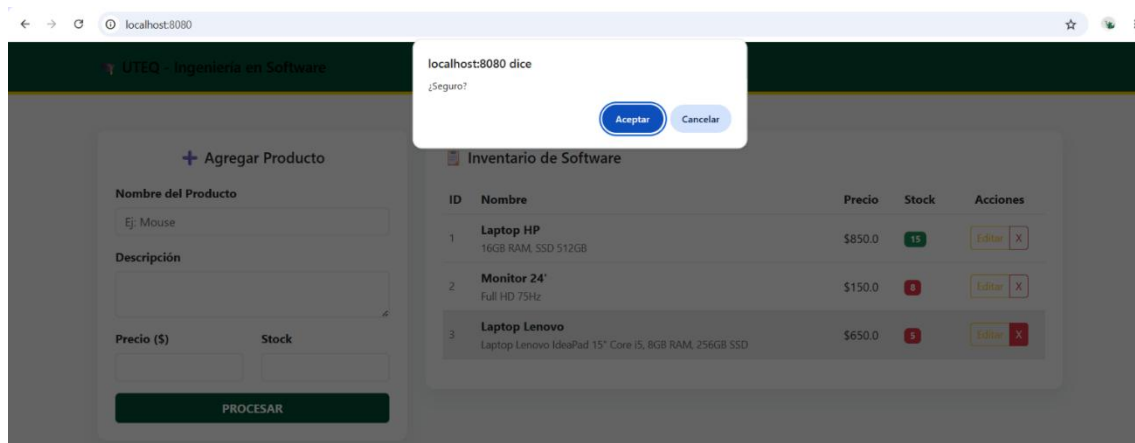
```
@PostMapping("/guardar")
public String guardar(Product p) {
    // LOG REPETIDO
    System.out.println("[DEBUG LOG] Guardando producto: " +
p.getNombre() + " - Fecha: " + new java.util.Date());

    // Lógica mezclada
    if (p.getId() == null) {
        p.setId(idCounter++);
        productos.add(p);
    } else {
        for (Product item : productos) {
            if (item.getId().equals(p.getId())) {
                item.setNombre(p.getNombre());
                item.setDescripcion(p.getDescripcion());
                item.setPrecio(p.getPrecio());
                item.setStock(p.getStock());
            }
        }
    }
    return "redirect:/";
}
```

Interfaz

The screenshot shows a web application interface with a dark green header bar containing the text "UTEQ - Ingeniería en Software". Below the header, there are two main panels. The left panel, titled "Editar Producto", contains a form with the following fields: "Nombre del Producto" (with the value "Impresora Epson"), "Descripción" (with the value "Impresora multifuncional WiFi con sistema de tinta continua"), "Precio (\$)" (with the value "220,0"), and "Stock" (with the value "8"). At the bottom of the form are two buttons: "PROCESAR" (in a dark green box) and "Cancelar edición" (in a light gray box). The right panel, titled "Inventario de Software", displays a table with the following data:

ID	Nombre	Precio	Stock	Acciones
1	Laptop HP 16GB RAM, SSD 512GB	\$850.0	15	Editar X
2	Monitor 24" Full HD 75Hz	\$150.0	1	Editar X
3	Laptop Lenovo Laptop Lenovo IdeaPad 15" Core i5, 8GB RAM, 256GB SSD	\$650.0	5	Editar X
4	Impresora Epson Impresora multifuncional WiFi con sistema de tinta continua	\$220.0	1	Editar X



Conclusión:

En conclusión, el estado actual del proyecto presenta una alta deuda técnica debido a la falta de modularidad. La acumulación de responsabilidades en una sola clase (God Object) y la redundancia de funciones (Código Duplicado) no solo aumentan la probabilidad de errores ante futuros cambios, sino que también impiden la escalabilidad del sistema. Este diseño demuestra que, aunque la aplicación es funcional, su estructura es frágil y costosa de mantener, lo que evidencia la necesidad urgente de una refactorización bajo patrones de diseño profesionales.